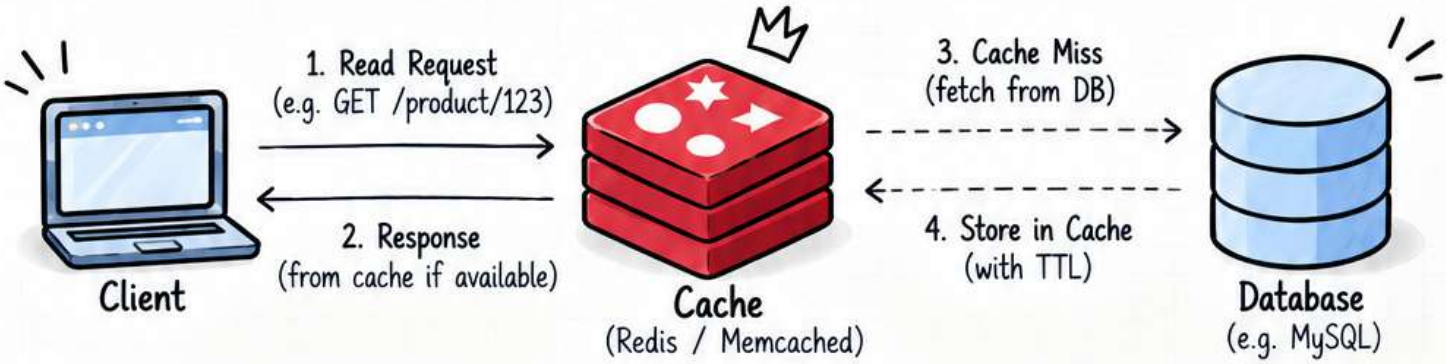
Concepts
Diagrams
Examples
Best Practices

Caching

- [x] Database Caching
- [x] Application Caching
- [x] API Caching
- [x] Distributed Caching
- [x] Consistency & Invalidation
- [x] Eviction Policies
- [x] TTL and more...

Cheat Sheet + 100 Interview Questions

From Basics to Advanced | With Real World Examples | For SDE & System Design Interviews



1. Application Level Caching
2. API Caching + REST API Caching
3. Cache Aside Read Through Write Through
4. Write Back + Write Around
5. Cache Invalidation + TTL
6. Cache Eviction Policies + LRU, LFU
7. Distributed Caching + Cache Consistency
8. Caching Cheat Sheet
9. 100 Interview Questions
10. Best Practices + Real World Examples

Topics Covered

- Types of Caching
- Caching Patterns
- Caching Strategies
- Cache Invalidation
- TTL Strategies
- Cache Eviction (LRU, LFU)
- Distributed Caching
- Cache Consistency
- Real World Examples
- 100 Interview Questions + Answers

Concepts to Interview Ready?

Tech Stack & Tools

- Redis
- Memcached
- AWS ElastiCache
- GCP Memorystore
- Azure Cache for Redis

@Codedsoul_05

1. Caching Fundamentals + Why Caching ?

What is Caching ?

Caching is the process of storing frequently accessed data in a fast and temporary storage layer (cache) so that it can be served quickly without fetching it from the original source every time. It helps in reducing response time and improves the overall system performance.

Why Caching is Needed ?

In modern applications, data is requested again and again (e.g. user profiles, product details, API responses). Fetching the same data from the database every time is slow and puts heavy load on the database. Caching helps serve this frequently accessed data from a faster storage layer.

Benefits of Caching

- Lower latency (Faster response time)
- Reduces database load
- Higher throughput (Handle more requests)
- Better scalability (Support more users)
- Improved user experience
- Efficient use of resources

Common Cache Locations

Browser Cache	Stores static assets
CDN Cache	Caches content at edge
Application Cache	In-memory cache (e.g. Redis)
Database Cache	Query result cache
OS / Server Cache	File system, memory cache

Basic Request Flow with Caching



Cache Hit vs Cache Miss

Cache Hit	Requested data is found in cache and served directly. (Fast)
Cache Miss	Requested data is not found in cache, so it is fetched from the database and then stored in cache. (Slower)

Commonly Cached Data

- User profile information
- Product details
- API responses
- Session data
- Static files (images, CSS, JS)
- Database query results
- Configuration data

Time To Live (TTL) Basics

TTL (Time To Live) is the expiration time for a cached item. After the TTL expires, the data is removed from cache and a new request will fetch fresh data from the source.

Key Trade-offs / Challenges

- Stale data (Cache may have outdated data)
- Additional memory cost (Requires extra storage)
- Cache invalidation complexity (When to update or remove data)
- Consistency issues (Keeping cache and database in sync)
- Not suitable for rarely accessed data

Note :
Caching is a trade-off between speed and freshness of data.

Key Takeaways

- Caching stores frequently accessed data in a fast storage layer.
- It reduces latency, database load and improves scalability.
- Choose the right cache location and TTL based on your use case.
- Handle invalidation and consistency carefully.

2. Browser Cache + HTTP Caching

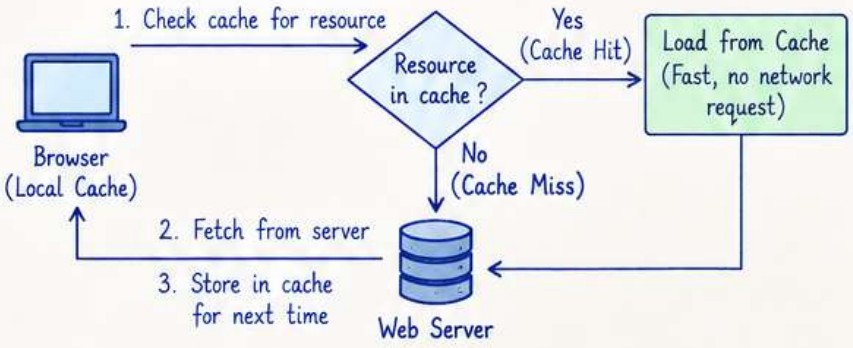
What is Browser Cache ?

Browser cache stores static resources (HTML, CSS, JS, images, fonts, etc.) on the user's local device so that they can be loaded faster on subsequent visits without making a network request to the server.

What is HTTP Caching ?

HTTP caching uses response headers to control how resources are cached by the browser or intermediate caches (like CDN and proxy). It tells for how long a resource can be stored and when it should be revalidated.

How Browser Caching Works ?



Types of Caching in Browser

Memory Cache	Stores resources in RAM. Fastest but cleared when tab is closed.
Disk Cache	Stores resources on disk. Persistent even after browser restart.
Service Worker Cache	Used in Progressive Web Apps (PWA) for offline access.
HTTP Cache	Controlled using HTTP headers (e.g. Cache-Control, ETag, etc.).

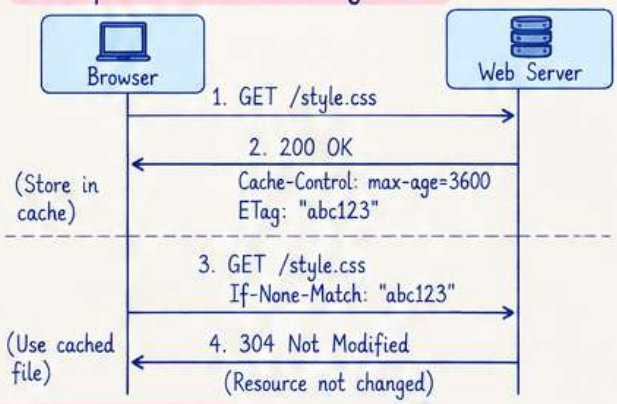
HTTP Caching Headers

Header	Purpose	Example	Explanation
Cache-Control	Specifies caching rules	Cache-Control: max-age=3600	Resource can be cached for 3600 seconds (1 hour)
Expires	Specifies expiry date	Expires: Wed, 21 Oct 2026 07:28:00 GMT	Resource is valid until this date
ETag	Resource version identifier	ETag: "abc123"	Used for validation (checks if resource changed)
Last-Modified	Last modified timestamp	Last-Modified: Wed, 21 Oct 2026 07:00:00 GMT	Used for validation with If-Modified-Since
If-None-Match	Sent by browser for validation	If-None-Match: "abc123"	Server returns 304 if not changed
If-Modified-Since	Sent by browser for validation	If-Modified-Since: Wed, 21 Oct 2026 07:00:00 GMT	Server returns 304 if not changed

Cache-Control Directives (Important)

Directive	Meaning
max-age	Time (in seconds) to cache the resource
no-cache	Must revalidate with server before using
no-store	Do not store in cache
public	Can be cached by browser and intermediate caches
private	Can be cached only by browser
must-revalidate	Must revalidate after expiry
immutable	Resource will never change (useful for versioned files)

Example: HTTP Caching Flow



Cache Hit vs Cache Miss (HTTP)

Cache Hit	Cache Miss
Resource found in browser cache	Resource not found or expired
No network request (or 304)	Browser makes request to server
Faster response time	Slower response time
Reduces server load	New resource is stored in cache

Best Practices

- ✅ Use long cache duration for static files (CSS, JS, images)
- ✅ Use versioning or hash in file names (app.v1.2.3.js)
- ✅ Use Cache-Control and ETag together
- ✅ Use immutable for versioned assets
- ✅ Use no-store for sensitive data (user information)
- ✅ Leverage CDN for static assets
- ✅ Properly handle cache invalidation when content changes

3. CDN Caching + Edge Caching

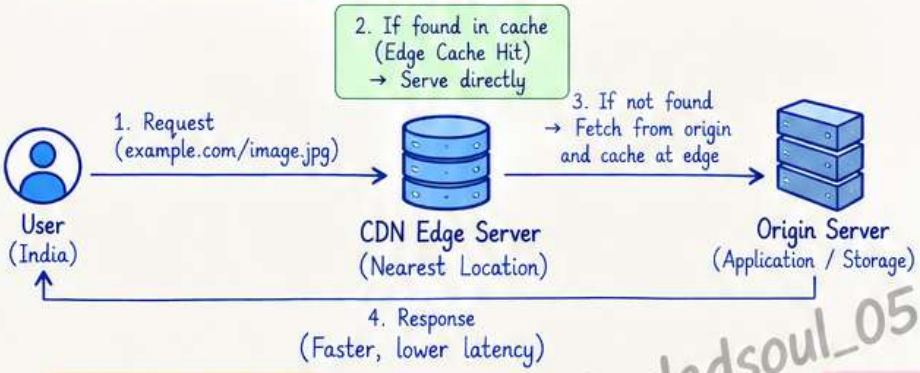
What is CDN ?

CDN (Content Delivery Network) is a global network of servers that delivers static and cached content (images, CSS, JS, videos, etc.) to users from the nearest location, reducing latency and improving load time.

What is Edge Caching ?

Edge caching stores content at servers located closer to end users (edge locations). It is a part of CDN where content is cached at the edge to serve requests faster and reduce the load on the origin server.

How CDN + Edge Caching Works ?



Benefits of CDN / Edge Caching

- ✅ Lower latency (faster content delivery)
- ✅ Reduces load on origin server
- ✅ Improves website performance (Page Speed)
- ✅ Handles high traffic and global users
- ✅ Better availability and reliability
- ✅ DDoS protection and security
- ✅ Scales automatically
- ✅ Cost efficient for large static content

@Codedsoul_05

CDN Components

Edge Location	Servers placed globally to cache content
Origin Server	Original server where content lives
Cache	Stores frequently accessed content
DNS	Routes user to nearest edge server
CDN Provider	Manages global infrastructure (e.g. Cloudflare, Akamai, AWS CloudFront)

Types of Content Cached in CDN

Images (JPG, PNG, WebP)	CSS Files	JavaScript Files
Videos (MP4, HLS)	Fonts (WOFF, WOFF2)	Static API Responses (JSON, XML)

CDN vs Edge Caching vs Normal Hosting

Feature	Normal Hosting	CDN Caching	Edge Caching
Server Location	Single region	Multiple global locations	Nearest edge locations
Content Storage	No caching	Caches static content	Caches at edge (closest to user)
Latency	Higher	Lower	Very low
Scalability	Limited	High	Very high
Best For	Dynamic content	Static content	Static + Some dynamic content (edge functions)

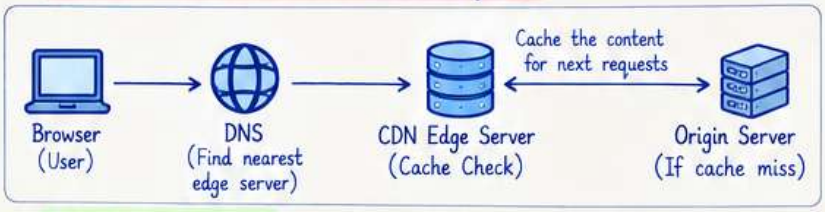
Popular CDN Providers

Cloudflare · AWS CloudFront · Azure CDN · Akamai · Fastly

Common Use Cases

- Serving static assets (images, CSS, JS)
- Video streaming and media delivery
- Global content delivery for websites and apps
- Caching API responses (with proper headers)
- E-commerce and high traffic websites
- Software downloads and updates

Cache Flow with CDN (Example)



Key Headers for CDN Caching

Header	Purpose
Cache-Control	Defines how long content is cached
Expires	Specifies expiry date
ETag	Resource version for validation
Last-Modified	Timestamp for validation
Surrogate-Control	Used by CDNs for caching rules
CDN-Cache-Status	Shows cache hit or miss (e.g. HIT, MISS)

Key Takeaways

- CDN and edge caching significantly reduce latency.
- Store static and frequently accessed content at edge locations.
- Use proper caching headers to control cache behavior.
- Choose a CDN provider based on your use case and global reach.
- Edge functions can also be used for dynamic content in modern CDNs (e.g. Cloudflare Workers).

4. Server Side Caching

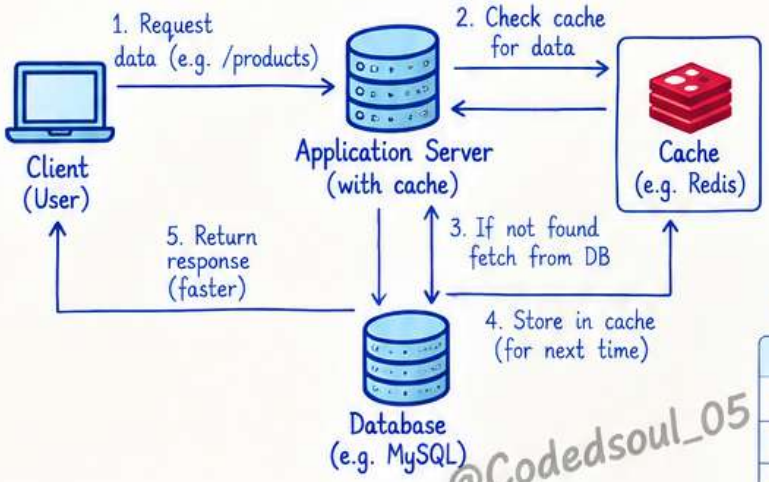
What is Server Side Caching ?

Server side caching stores frequently accessed data on the server (in memory, file system, or distributed cache like Redis) so that it can be served directly without hitting the database or performing expensive computations every time.

Why Server Side Caching is Needed ?

- Reduces database load
- Improves response time and throughput
- Handles high traffic and concurrent users
- Scales the application efficiently
- Helps in cost optimization
- Useful for data that is frequently read and rarely changes

How Server Side Caching Works ?



@Codedsoul_05

Where Server Side Caching is Used ?

- Caching API responses
- Caching database query results
- Storing user session data
- Caching frequently accessed objects (products, users, configurations)
- Caching computed results (reports, analytics)
- Caching pages in server side rendering (SSR)
- Used in microservices and distributed systems

Types of Server Side Caching

Type	Description
In Memory Caching	Stores data in RAM (e.g. Redis, Memcached)
Application Cache	Built-in cache in application (e.g. Node.js, Django)
Database Query Cache	Caches query results to reduce DB load
File System Cache	Stores files, templates, static content on server
Distributed Cache	Shared cache across multiple servers (e.g. Redis)

Server Side Caching vs Client Side Caching

Feature	Server Side Caching	Client Side Caching
Location	On server	On client/browser
Data Access	Shared for all users	Specific to user/device
Control	Full control for application	Limited control
Use Cases	API responses, DB queries, sessions, computed data	Static assets (CSS, JS, images)
Invalidation	Managed by server	Managed by browser

Popular Server Side Caching Tools

Redis	Memcached	Ehcache	Varnish Cache	Database Cache
(In-memory, most popular)	(Simple & lightweight)	(Java applications)	(HTTP accelerator)	(MySQL, PostgreSQL)

Cache Strategies (For Server Side)

Strategy	Description
Cache Aside (Lazy Loading)	Application checks cache, if not found, fetches from DB and stores in cache.
Read Through	Cache handles data loading from DB automatically.
Write Through	Data is written to cache and DB at the same time.
Write Back (Write Behind)	Data is written to cache first and later to DB.
Write Around	Writes go directly to DB, cache is used only for reads.

Example: Caching API Response (Node.js + Redis)

```
const redis = require('redis');
const client = redis.createClient();

app.get('/products', async (req, res) => {
  const cacheKey = 'products';
  const cachedData = await client.get(cacheKey);

  if (cachedData) {
    return res.json(JSON.parse(cachedData)); // Return from cache
  }

  // If not found, fetch from DB
  const data = await getProductsFromDB();
  await client.setex(cacheKey, 3600, JSON.stringify(data)); // Cache for 1 hour
  return res.json(data);
});
```

Key Takeaways

- ✅ Server side caching reduces latency and database load.
- ✅ Use the right caching strategy based on the use case.
- ✅ Redis is widely used for scalable server side caching.
- ✅ Cache invalidation is important to maintain data consistency.
- ✅ It improves performance and scalability of applications.

Common Challenges

- ❌ Cache invalidation complexity
- ❌ Stale data (outdated information)
- ❌ Memory usage and cost
- ❌ Distributed cache consistency
- ❌ Choosing the right expiration time (TTL)

5. In Memory Caching

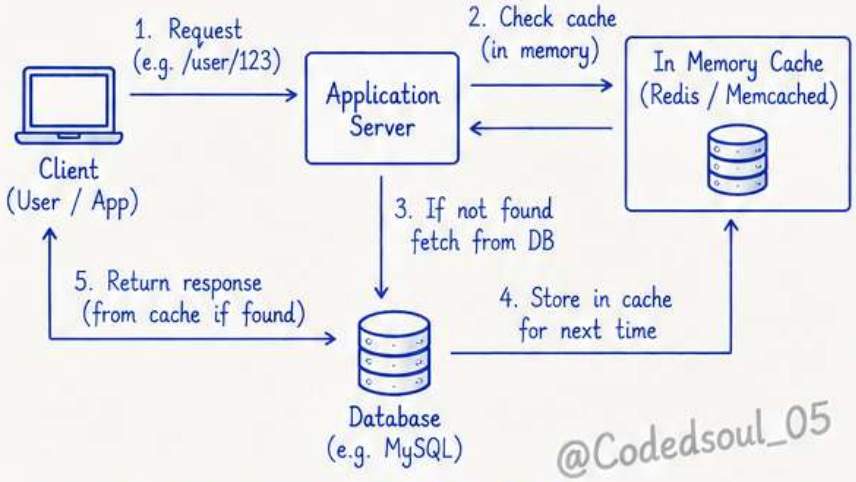
What is In Memory Caching ?

In memory caching stores data in the RAM of a server instead of disk, so that frequently accessed data can be retrieved very quickly with low latency.

Why In Memory Caching is Needed ?

- Faster data access (low latency)
- Reduces database load
- Handles high traffic and concurrent users
- Improves application performance
- Useful for frequently read and rarely changing data
- Supports real time applications

How In Memory Caching Works ?



@Codedsoul_05

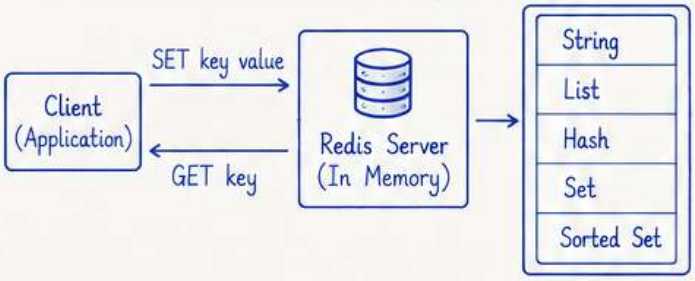
Characteristics

Storage Location	RAM (Memory)
Access Speed	Very Fast (microseconds)
Data Type	Key - Value pairs, objects, strings, lists, sets, hashes
Persistence	Usually temporary (some tools support persistence)
Use Case	Frequently accessed and real time data
Examples	Redis, Memcached, Ehcache, Caffeine

Popular In Memory Caching Tools

Tool	Description
Redis	Open source, rich data structures, supports persistence, most popular
Memcached	Simple, high performance, key - value store
Ehcache	Popular in Java applications
Caffeine	High performance caching for Java (in-process)

Redis Architecture (Simple View)



Common Data Structures in Redis

Data Structure	Use Case
String	Store simple key - value (e.g. user session)
List	Queues, recent activities
Hash	Store object / multiple fields (e.g. user data)
Set	Unique items (e.g. online users)
Sorted Set	Leaderboard, ranking
Bitmap	Store 0/1 values (e.g. user activity)
HyperLogLog	Count unique elements (e.g. unique visitors)

In Memory Caching vs Other Caching

Feature	In Memory Caching	Database Caching	CDN Caching
Storage	RAM	DB memory / cache	Edge servers
Speed	Very Fast	Fast	Fast (global)
Scope	Application level	Database level	Global (static content)
Use Case	Dynamic data (session, objects)	Query results	Static files (images, CSS, JS)
Examples	Redis, Memcached	MySQL Query Cache	Cloudflare, Akamai

Advantages

- Very low latency
- Reduces database load
- Supports high concurrency
- Improves application performance
- Flexible data structures
- Widely used in real time systems

Disadvantages

- Data is stored in RAM (higher cost)
- Limited memory size
- Data is mostly temporary (can be lost on restart)
- Requires proper cache invalidation
- Needs monitoring and management

Key Takeaways

- In memory caching is used to store frequently accessed data in RAM.
- It provides very fast access and improves application performance.
- Redis and Memcached are the most popular tools.
- Choose the right tool and strategy based on your use case.

6. Redis + Memcached

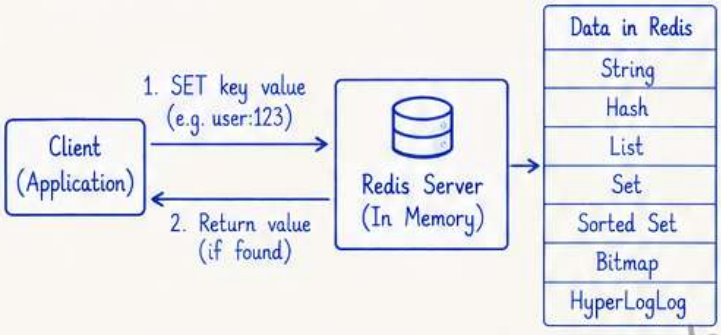
What is Redis ?

Redis (Remote Dictionary Server) is an open source, in-memory data store that supports various data structures like strings, hashes, lists, sets, sorted sets, etc. It is widely used for caching, session storage, real time analytics and message queues.

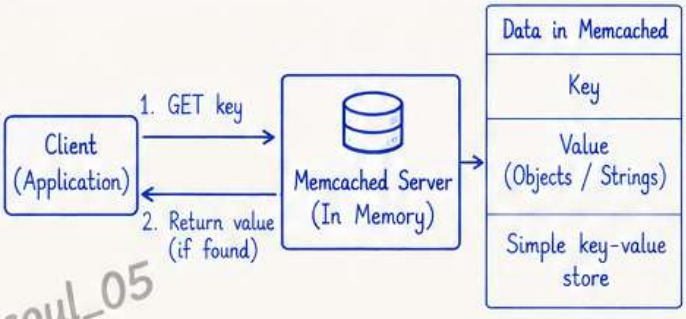
What is Memcached ?

Memcached is a high performance, in-memory key-value store used primarily for caching simple data such as objects, API responses and database query results. It is simple, lightweight and optimized for speed.

How Redis Works ?



How Memcached Works ?



@Codedsoul_05

Key Features of Redis

Feature	Description
In Memory	Stores data in RAM (very fast)
Rich Data Structures	String, Hash, List, Set, Sorted Set, Bitmap, HyperLogLog, Streams
Persistence	Supports RDB and AOF (optional)
Replication	Master-Slave replication
Clustering	Supports Redis Cluster (horizontal scaling)
Pub/Sub	Real time messaging
Transactions	Supports atomic operations using MULTI/EXEC
Use Cases	Caching, session storage, real time analytics, rate limiting, leaderboards

Key Features of Memcached

Feature	Description
In Memory	Stores data in RAM (very fast)
Simple Key-Value	Only supports simple key-value pairs
Lightweight	Minimal feature set, easy to use
No Persistence	Data is lost on restart
Multi-threaded	Handles multiple connections efficiently
Auto Sharding	Client side sharding for distributed caching
Use Cases	Caching database queries, API responses, web page fragments, session data

Data Structures in Redis

Data Structure	Description	Example
String	Simple key-value pair	SET name "Nikhil"
Hash	Store multiple fields	HSET user:1 name Nikhil
List	Ordered collection	LPUSH tasks "task1"
Set	Unique values	SADD users 1 2 3
Sorted Set	Sorted with score	ZADD leaderboard 100 "Nikhil"
Bitmap	Store 0/1 values	SETBIT visits 1 1
HyperLogLog	Count unique elements	PFADD visitors user1
Streams	Log style data	XADD logs * level info

Redis vs Memcached (Comparison)

Feature	Redis	Memcached
Data Model	Rich data structures	Simple key-value
Supported Types	String, Hash, List, Set, Sorted Set, etc.	Only Strings / Objects
Persistence	Yes (RDB, AOF)	No
Replication	Yes	No (Handled by client)
Clustering	Yes (Redis Cluster)	Limited (Client side)
Performance	Very high	Very high
Memory Usage	Higher	Lower
Complexity	More features	Simple and lightweight
Use Cases	Caching, real time data, sessions, queues	Simple caching of objects and queries

When to Use Redis ?

- When you need complex data structures.
- When data persistence is important.
- For real time data, analytics, messaging, or queues.
- When you need high availability and replication.
- When you need caching with advanced features.

When to Use Memcached ?

- When you need simple and fast caching.
- When persistence is not required.
- For caching database queries or API responses.
- When you want lightweight and easy setup.
- When you have simple key-value use cases.

Key Takeaways

- Redis is more powerful and feature rich.
- Memcached is simpler and faster for basic caching.
- Choose based on your use case and requirements.

7. Database Query Caching

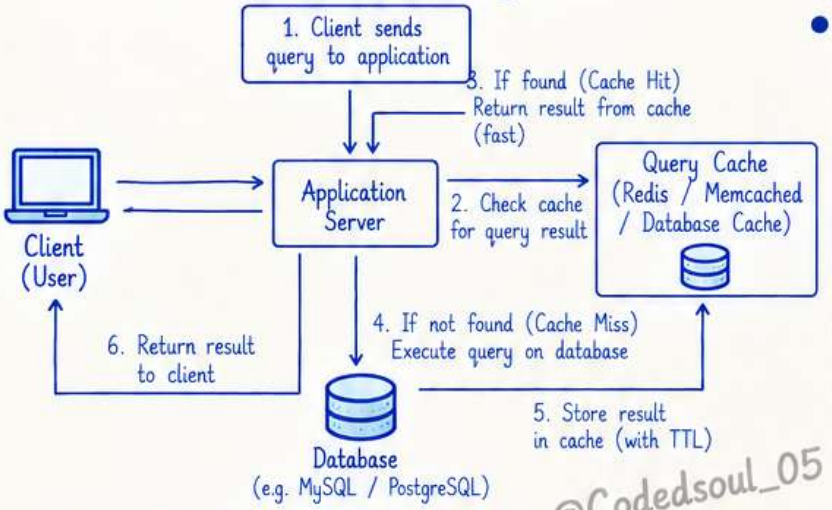
What is Database Query Caching ?

Database query caching stores the result of frequently executed queries in memory so that subsequent identical queries can be served faster, reducing the load on the database and improving application performance.

Why Database Query Caching is Needed ?

- Reduces database load
- Improves query response time
- Handles high traffic and repeated queries
- Saves database resources (CPU, I/O)
- Useful for read heavy applications
- Improves overall application performance and scalability

How Database Query Caching Works ?



What Can Be Cached ?

Data Type	Examples
Query Results	SELECT * FROM users WHERE id = 1
Aggregated Data	COUNT(*), SUM(), AVG()
Reference Data	Countries, Categories, Settings
API Data (from DB)	Product list, user profile
Expensive Joins	Complex queries with multiple tables
Reports / Analytics	Dashboard data, charts
Metadata	Schema info, configuration data

@Codedsoul_05

Types of Database Query Caching

Type	Description
Application Level Caching	Cache query results in application using Redis, Memcached, etc.
Database Level Caching	Built in caching provided by database (e.g. MySQL query cache - deprecated)
ORM Level Caching	Caching provided by ORM (e.g. Hibernate, Django ORM cache)
Materialized Views	Precomputed and stored query results
Result Set Caching	Cache complete result set of a query
Query Plan Caching	Cache execution plan for faster query execution

Cache Invalidation Strategies

Strategy	Description
TTL Based	Automatically expire cache after a time (e.g. 5 minutes, 1 hour)
Event Based	Invalidate cache when data is updated
Manual Invalidation	Clear cache explicitly when needed
Versioning	Use version or timestamp in cache key
Write Through	Update cache whenever data is written
Write Back	Update cache first and database later
Cache Aside	Application checks cache, if not found fetch from DB and store in cache

Example: Query Caching with Redis (Node.js)

```
const redis = require('redis');
const client = redis.createClient();

async function getUserById(id) {
  const cacheKey = `user:${id}`;

  // Check cache
  const cachedData = await client.get(cacheKey);
  if (cachedData) {
    return JSON.parse(cachedData); // Return from cache
  }

  // Fetch from database
  const user = await db.query('SELECT * FROM users WHERE id = ?', [id]);

  // Store in cache for 10 minutes (600 seconds)
  await client.setEx(cacheKey, 600, JSON.stringify(user[0]));

  return user[0];
}
```

Best Practices

- ✅ Cache only frequently executed and expensive queries
- ✅ Use appropriate TTL based on data freshness
- ✅ Use cache invalidation strategy (event based or TTL)
- ✅ Do not cache sensitive or highly dynamic data
- ✅ Use Redis for distributed caching in multiple servers
- ✅ Monitor cache hit ratio and performance
- ✅ Use proper cache key naming (e.g. user:1, products:page:2)
- ✅ Avoid caching large result sets
- ✅ Combine with indexing and query optimization

Advantages

- ✅ Faster query response time
- ✅ Reduces database load
- ✅ Improves application performance
- ✅ Scales better for high traffic
- ✅ Saves database resources
- ✅ Useful for read heavy operations

Disadvantages

- ❌ Stale data if not invalidated properly
- ❌ Extra memory usage
- ❌ Cache management complexity
- ❌ Not suitable for highly dynamic data
- ❌ Can increase system complexity
- ❌ Need to monitor cache consistency

Common Use Cases

- User profiles and session data
- Product catalog and listings
- Dashboard and analytics data
- Frequently accessed reference data
- API responses that query database
- E-commerce product details
- Reports and aggregated data

8. Application Level Caching

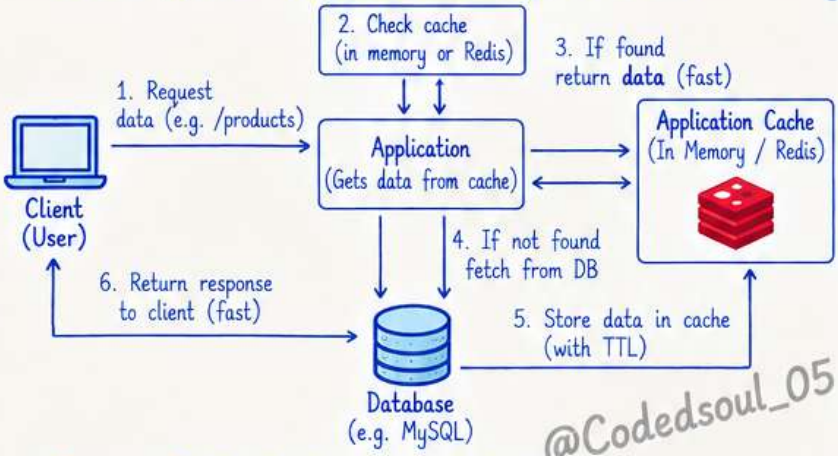
What is Application Level Caching ?

Application level caching is a caching mechanism implemented inside the application code to store frequently accessed data (objects, API responses, computed results, etc.) in memory or an external cache (like Redis, Memcached) so that it can be retrieved quickly without hitting the database or external services.

Why Application Level Caching is Needed ?

- Reduces database and external API load
- Improves application response time
- Handles high traffic and concurrent users
- Saves computational resources
- Useful for data that is frequently read and rarely changes
- Improves overall system performance and scalability

How Application Level Caching Works ?



@Codedsoul_05

Where Application Level Caching is Used ?

Use Case	Example
API Response Caching	Cache response of frequently called APIs
User Session Data	Store user session information
Computed Results	Cache expensive computations
Static Data	Configuration, feature flags, reference data
Page Fragments	Cache rendered HTML fragments (in SSR applications)
Third Party API Data	Cache responses from external APIs (like payment, weather, maps)

Types of Application Level Caching

Type	Description
In Memory Caching	Stores data in application memory (e.g. HashMap, ConcurrentMap). Fast but limited to single server.
Distributed Caching	Uses external cache like Redis or Memcached. Shared across multiple servers.
Local Cache	Cache stored in the same application instance (e.g. Caffeine, Guava).
Session Cache	Caches user session data in application.
Object Cache	Caches frequently used objects (e.g. product details).
Fragment Cache	Caches rendered views or page fragments.
API Cache	Caches external API responses.

In Memory Cache vs Distributed Cache

Feature	In Memory Cache	Distributed Cache
Storage Location	Application memory	External server (e.g. Redis)
Scope	Single application instance	Multiple application instances
Speed	Very fast	Fast
Data Sharing	Not shared	Shared across servers
Scalability	Limited	High
Persistence	Lost on application restart	Can be configured for persistence
Examples	HashMap, Caffeine, Guava	Redis, Memcached

Example: Application Level Caching (Node.js + Redis)

```
const redis = require('redis');
const client = redis.createClient();

async function getProduct(id) {
  const cacheKey = `product:${id}`;

  // Check cache
  const cachedData = await client.get(cacheKey);
  if (cachedData) {
    return JSON.parse(cachedData); // Return from cache
  }

  // If not found, fetch from database
  const product = await db.query('SELECT * FROM products WHERE id = ?',[id]);

  // Store in cache for 10 minutes (600 seconds)
  await client.setEx(cacheKey, 600, JSON.stringify(product));
  return product;
}
```

Popular Application Level Caching Tools

Tool	Description
Redis	Distributed in-memory cache, most popular
Memcached	Simple, high performance, key-value store
Caffeine	High performance in-memory cache for Java
Guava Cache	In-memory cache library for Java
Ehcache	Java caching framework
Django Cache	Built-in caching system in Django
Spring Cache	Caching abstraction in Spring Boot
Node Cache	Simple in-memory cache for Node.js

Advantages

- ✅ Faster response time
- ✅ Reduces database and external API load
- ✅ Improves application performance
- ✅ Handles high traffic and concurrency
- ✅ Simple to implement
- ✅ Supports various types of data

Disadvantages

- ❌ Memory usage increases
- ❌ Requires cache invalidation
- ❌ Stale data if not managed properly
- ❌ Adds complexity to application code
- ❌ In-memory cache is lost on restart
- ❌ Needs monitoring and tuning

Key Takeaways

- Use application level caching for frequently accessed and rarely changing data.
- Choose the right cache tool (in-memory or distributed).
- Set proper TTL and eviction strategy.
- Implement cache invalidation for data consistency.
- Monitor cache hit ratio, memory usage and performance.
- Combine with database and CDN caching for best results.

9. API Caching + REST API Caching

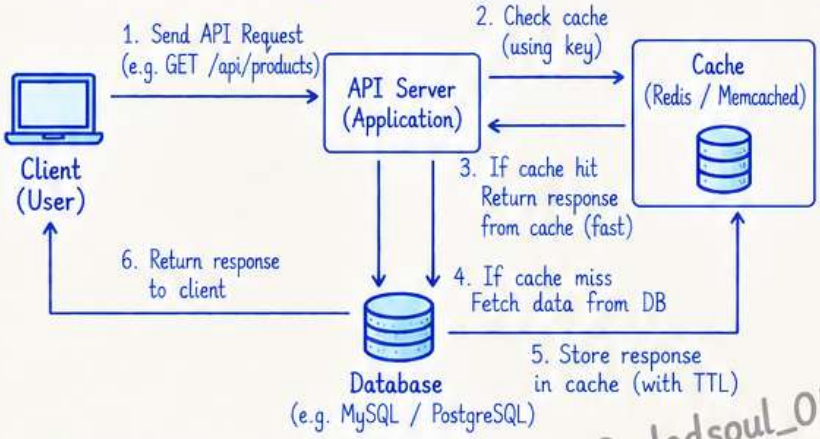
What is API Caching ?

API caching stores the response of frequently called APIs in cache (in memory or distributed) so that the same response can be served quickly for subsequent requests without executing the entire business logic or database queries again.

Why API Caching is Needed ?

- Reduces response time (low latency)
- Reduces server and database load
- Handles high traffic and concurrent users
- Saves computational resources
- Improves scalability and user experience
- Useful for data that is frequently read and rarely changes

How API Caching Works ?



What Can Be Cached in APIs ?

Data Type	Example
API Response	GET /products, GET /users
List Data	Product list, category list
Reference Data	Countries, cities, settings
Computed Data	Aggregated results, dashboard data
Static Content	Config, feature flags, metadata
External API Data	Weather API, Payment API, Maps API
User Specific Data	User profile, preferences (with proper key)

@Codedsoul_05

REST API Caching Methods

Method	Description
In Memory Cache	Store API response in application memory (e.g. HashMap, Caffeinie).
Distributed Cache	Use Redis or Memcached for shared cache across multiple servers.
HTTP Caching	Use cache headers (e.g. Cache-Control, ETag).
Reverse Proxy Cache	Use CDN or proxy server (e.g. Nginx, Varnish).
Cache Middleware	Use framework middleware (e.g. Express, Django).
Client Side Cache	Cache API response in browser or mobile app.

HTTP Caching for REST APIs

Header	Description
Cache-Control	Defines caching rules (max-age, no-cache, etc.)
Expires	Specifies expiry time for cached response
ETag	Unique identifier for resource (used for validation)
Last-Modified	Timestamp of last modification
If-None-Match	Used with ETag for conditional requests
If-Modified-Since	Used with Last-Modified for conditional requests

Example: API Caching with Redis (Node.js + Express)

```
const express = require('express');
const redis = require('redis');
const app = express();
const client = redis.createClient();

app.get('/api/products', async (req, res) => {
  const cacheKey = 'products:list';

  // Check cache
  const cachedData = await client.get(cacheKey);
  if (cachedData) {
    return res.json(JSON.parse(cachedData)); // Return from cache
  }
  // Fetch from database
  const products = await db.query('SELECT * FROM products');

  // Store in cache for 10 minutes
  await client.setEx(cacheKey, 600, JSON.stringify(products));
  return res.json(products);
});
```

Cache Keys for APIs

Key Type	Example
Simple Key	products:list
With Parameters	products:category:electronics
User Specific	user:123:profile
Pagination	products:page:2:limit:10
With Filters	products:search:phone:sort:price
Versioned	v1:products:list

Cache Invalidation Strategies

Strategy	Description
TTL Based	Automatically expire after fixed time
Manual Invalidation	Clear cache when data is updated
Event Based	Invalidate on database change (publish/subscribe)
Versioning	Use version in key (e.g. v2:products:list)
Cache Busting	Change key when data changes

Advantages

- ✅ Faster API response time
- ✅ Reduces server and database load
- ✅ Improves scalability and performance
- ✅ Handles high traffic and concurrent users
- ✅ Saves computational resources
- ✅ Better user experience

Disadvantages

- ❌ Stale data if not invalidated properly
- ❌ Requires cache management
- ❌ Adds complexity to the system
- ❌ Memory usage and cost (for large data)
- ❌ Not suitable for frequently changing data
- ❌ Needs monitoring and tuning

Best Practices

- ✅ Cache only GET requests
- ✅ Use appropriate TTL based on data type
- ✅ Use descriptive cache keys
- ✅ Implement proper cache invalidation
- ✅ Do not cache sensitive data (e.g. passwords)
- ✅ Monitor cache hit ratio and performance

10. Cache Aside + Read Through + Write Through

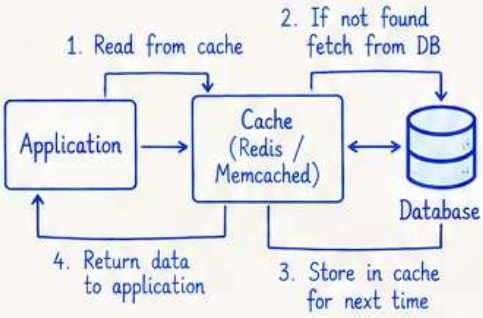
What are Caching Patterns ?

Caching patterns define how the application interacts with the cache and the database for reading and writing data. The most commonly used patterns are Cache Aside, Read Through and Write Through.

Why Different Caching Patterns ?

- Different applications have different read write needs
- Helps to balance performance, data consistency and complexity
- Avoids cache stampede and stale data issues
- Useful for designing scalable and reliable systems

1. Cache Aside (Lazy Loading)

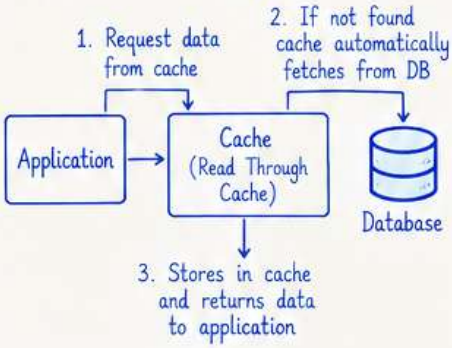


Flow for Read Operation

1. Application checks data in cache
2. If not found, fetch from database
3. Store the data in cache (with TTL)
4. Return data to application

When to Use	Read heavy applications
Data Freshness	Slightly stale data (acceptable)
Complexity	Simple
Popular Tools	Redis, Memcached

2. Read Through

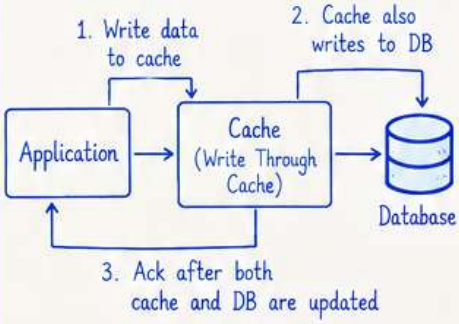


Flow for Read Operation

1. Application requests data from cache
2. If not found, cache layer automatically fetches data from database
3. Cache stores the data and returns it to application

When to Use	Read heavy applications with managed cache layer
Data Freshness	Slightly stale data
Complexity	Medium (handled by cache library)
Popular Tools	Ehcache, Hazelcast, Redis (with read through support)

3. Write Through



Flow for Write Operation

1. Application writes data to cache
2. Cache immediately writes the data to database
3. After both are updated, success is returned to application

When to Use	Write heavy applications where data consistency is important
Data Freshness	Always up to date
Complexity	Medium
Popular Tools	Redis, Hazelcast, Custom cache layer

@Codedsoul_05

Comparison of Caching Patterns

Feature	Cache Aside	Read Through	Write Through
Read Flow	App checks cache, then DB if not found	Cache handles fetching from DB automatically	Not applicable
Write Flow	Write to DB, then invalidate cache	Not applicable	Write to cache and DB together
Data Freshness	May be stale	Slightly stale	Always fresh
Read Performance	Fast after first load	Fast (managed by cache)	Fast
Write Performance	Fast	Fast	Slower (needs to write to cache and DB)
Complexity	Simple	Medium	Medium
Best Use Case	Read heavy systems with manual control	Applications using cache libraries or frameworks	Write heavy systems with strong consistency

Key Points to Remember

- Cache Aside is the most commonly used pattern in real world applications
- Read Through is useful when using a cache framework that supports automatic loading
- Write Through ensures strong consistency but adds write latency
- Choose the pattern based on your application read write ratio and data consistency requirements
- All patterns are often used with TTL and cache invalidation strategies

Advantages

- Improves application performance
- Reduces database load
- Handles high traffic efficiently
- Flexible patterns for different use cases
- Can be combined with other caching strategies like TTL and invalidation

Disadvantages

- May lead to stale data (Cache Aside and Read Through)
- Write Through can be slower
- Requires proper cache invalidation
- Adds additional complexity in architecture
- Needs monitoring to avoid cache consistency issues

Common Use Cases

- E-commerce product catalog (Read heavy)
- User session data (Read Through)
- Configuration data (Cache Aside)
- Real time analytics dashboards (Read heavy with Cache Aside)
- Financial systems (Write Through for strong consistency)

11. Write Back + Write Around Caching

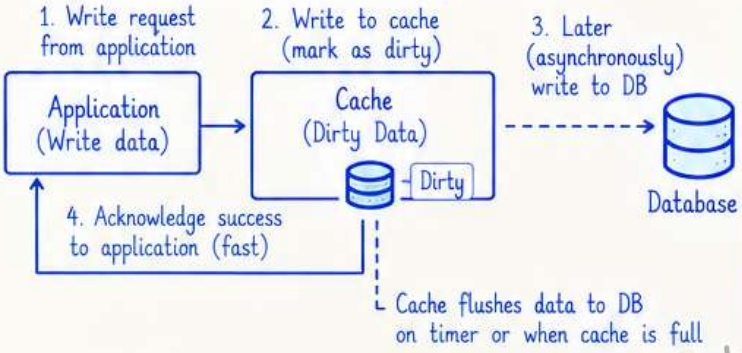
What is Write Back Caching ?

Write back caching (also called write behind) stores write operations in the cache first and updates the database later (asynchronously). Data is written to the database only after a certain time or when the cache is flushed.

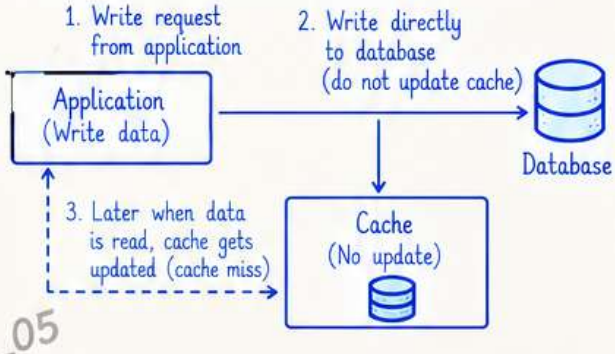
What is Write Around Caching ?

Write around caching bypasses the cache for write operations. The data is written directly to the database without updating the cache. Cache is only updated when the data is read later, preventing unnecessary cache pollution.

How Write Back Caching Works ?



How Write Around Caching Works ?



@Codedsoul_05

Flow for Write Back

1. Application writes data to cache.
2. Data is marked as dirty (not yet written to DB).
3. Data is written to database later (on timer or when cache is full).
4. Application gets immediate success response.

Flow for Write Around

1. Application writes data directly to database.
2. Cache is not updated.
3. When the data is read later, it is fetched from database and then stored in cache (cache miss).
4. Avoids unnecessary cache updates for data that might not be read again.

Key Characteristics

Feature	Write Back
Write Operation	Write to cache first, update DB later
Cache Update	Immediate (with dirty flag)
Database Update	Asynchronous (delayed)
Read Operation	Read from cache
Consistency	Eventually consistent
Write Latency	Low (fast)
Risk	Data loss if cache fails before flush
Best For	Write heavy applications
Examples	Redis (with persistence), Memcached (with custom logic), in memory caches

Key Characteristics

Feature	Write Around
Write Operation	Write directly to database
Cache Update	No update on write
Database Update	Immediate
Read Operation	Read from cache (if available)
Consistency	Strong consistency
Write Latency	Higher (goes to DB)
Risk	No risk of data loss from cache
Best For	Write heavy data that is rarely read
Examples	Used in databases, content management systems, logging data, analytics events

Advantages and Disadvantages

Type	Advantages	Disadvantages
Write Back	• Fast write performance • Reduces database load • Good for write heavy systems	• Risk of data loss if cache fails • Eventually consistent • Requires cache management (flush strategy)
Write Around	• Avoids cache pollution • Good for data rarely read • Maintains strong consistency • Simple to implement	• Higher write latency • Does not benefit from cache on immediate read • More database load for writes

When to Use ?

Type	Use Cases
Write Back	• Write heavy applications • Session data, real time analytics • Data where slight delay is acceptable • Systems that can handle eventual consistency
Write Around	• Data that is rarely read • Logging and audit data • Large media uploads • Use cases where cache pollution should be avoided • When strong consistency is required

Key Takeaways

- Write back caches data first and updates the database later.
- Write around writes directly to the database and does not update the cache.
- Use write back for high write performance and write around for data rarely read.
- Choose the strategy based on your application's read write ratio and consistency needs.

12. Cache Invalidation + TTL

What is Cache Invalidation ?

Cache invalidation is the process of removing or updating stale data from cache when the underlying data in the database changes. It ensures that users always get the latest and correct data instead of outdated (stale) data.

What is TTL (Time To Live) ?

TTL is a time based expiration mechanism for cached data. Each cache entry has a fixed lifetime after which it automatically expires and is removed from the cache. TTL helps in keeping cache data fresh and prevents stale data when explicit invalidation is not possible.

Why Cache Invalidation and TTL are Needed ?

- Keeps data consistent and up to date
- Prevents users from seeing stale or incorrect data
- Handles scenarios where data changes frequently
- Reduces manual cache management overhead
- Important for both read heavy and write heavy applications

When to Use Each ?

Scenario	Use Invalidation	Use TTL
Data changes frequently	Yes	Optional
Data rarely changes	Optional	Yes
Real time data (e.g. stock price)	Yes	Short TTL
Static data (e.g. countries list)	No	Long TTL
Unknown update pattern	Yes	Yes (as a fallback)

Cache Invalidation Strategies

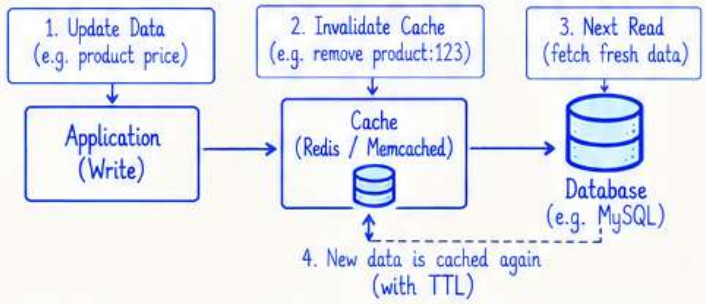
Strategy	Description	Use Case
Manual Invalidation	Explicitly remove or update cache when data changes	User profile update, product update
Event Based	Invalidate cache on events (e.g. DB update, message queue)	Microservices, distributed systems
Versioning	Use version in cache key (e.g. v1:user:123)	Avoid stale data without direct invalidation
Write Through	Update cache along with database	Strong consistency required
Write Back	Update cache first, write to DB later	High write performance

TTL Strategies

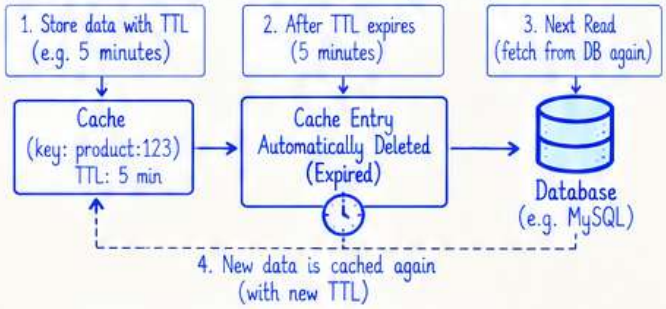
Strategy	Description	Example Use Case
Fixed TTL	Same TTL for all entries	5 minutes for product list
Dynamic TTL	Different TTL based on data type	Short TTL for stock price, long TTL for categories
Sliding TTL	Extend TTL on each access	Session data, user activity
Jittered TTL	Add random time to TTL to avoid cache stampede	5 minutes ± 30 seconds

@Codedsoul_05

How Cache Invalidation Works ? (Example Flow)



How TTL Works ? (Example Flow)



Example: Cache Invalidation in Node.js (Redis)

```
const redis = require('redis');
const client = redis.createClient();

// Update product in database
await db.query('UPDATE products SET price = ? WHERE id = ?', [999, 123]);

// Invalidate cache after update
await client.del('product:123');

// Next time, data will be fetched from DB and cached again with TTL
```

Example: TTL in Node.js (Redis)

```
const redis = require('redis');
const client = redis.createClient();

// Store data with TTL of 5 minutes (300 seconds)
await client.setEx('product:123', 300, JSON.stringify({
  id: 123,
  name: 'Laptop',
  price: 999
}));

// After 5 minutes, this key will automatically expire
```

Advantages and Disadvantages

Aspect	Advantages	Disadvantages
Cache Invalidation	• Keeps data consistent • Provides latest data • More control over caching	• Complex to implement • Needs event or manual triggers • Can be error prone in distributed systems
TTL	• Simple to implement • Automatic expiration • Reduces stale data • Works well for static/less frequent data	• Data can be stale until TTL expires • Not suitable for real time data • May increase database load after expiration • Choosing right TTL is challenging

Best Practices

- ✅ Use cache invalidation for frequently changing data
- ✅ Use TTL for static or less frequently changing data
- ✅ Combine invalidation with TTL as a fallback
- ✅ Use event based invalidation in distributed systems
- ✅ Use jittered TTL to prevent cache stampede
- ✅ Monitor cache hit ratio and stale data issues
- ✅ Choose TTL based on data type and update frequency
- ✅ Always test cache invalidation logic
- ✅ Use proper cache key naming with versioning if needed

13. Cache Eviction Policies + LRU, LFU

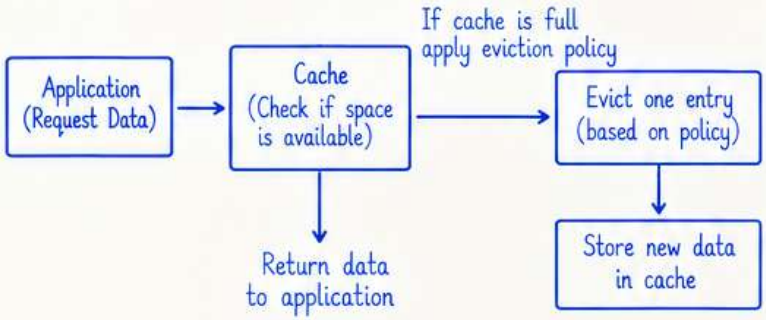
What is Cache Eviction ?

Cache eviction is the process of removing data from cache when the cache is full to make space for new data. It uses an eviction policy to decide which item to remove based on access patterns, frequency or time.

Why Eviction Policies are Needed ?

- Cache has limited memory
- New data needs to be stored when cache is full
- Helps to keep the most useful and frequently accessed data in cache
- Improves cache hit ratio and performance
- Avoids random removal of important data

How Cache Eviction Works ?



Common Cache Eviction Policies

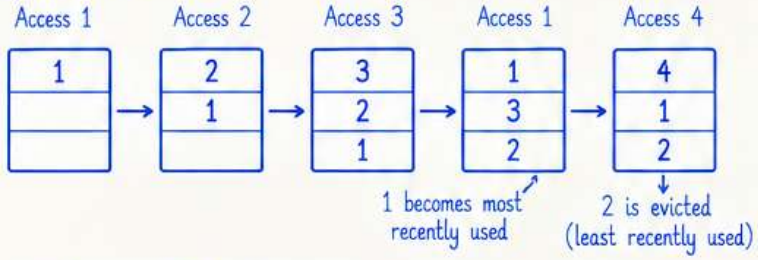
Policy	Idea	Good Use Case
LRU	Remove least recently used item	General purpose, most popular
LFU	Remove least frequently used item	When access frequency matters
FIFO	Remove first inserted item	Simple use cases
Random	Remove random item	Simple and fast
TTL Based	Remove items that are expired	Time sensitive data
Size Based	Remove items based on size	When items have different sizes

LRU (Least Recently Used)

- Removes the item that was not accessed for the longest time.
- Assumption: Recently accessed items are likely to be accessed again in the future.

Example:

Cache size = 3



LFU (Least Frequently Used)

- Removes the item with the lowest access frequency.
- Assumption: Items that are accessed more frequently are more likely to be used again.

Example:

Cache size = 3

Access	Cache State (key:frequency)	Evicted ?
1	1:1	-
2	1:1, 2:1	-
3	1:1, 2:1, 3:1	-
1	1:2, 2:1, 3:1	-
4	1:2, 4:1, 3:1	2 (lowest freq)

LRU vs LFU Comparison

Feature	LRU	LFU
Eviction Basis	Least recently used	Least frequently used
Tracks	Access time	Access frequency
Implementation	Simple (HashMap + Doubly Linked List)	Complex (HashMap + Min Heap)
Performance	Good in most cases	Better for stable access patterns
Handles Burst Access	Good	May keep old but frequently used items
Use Case	General purpose caching	When access frequency is important

Other Eviction Policies

Policy	Description	Example Use Case
FIFO	Remove the oldest inserted item	Simple caching, low complexity
Random	Remove a random item	When simplicity and speed are needed
TTL Based	Remove items after TTL expires	Session data, temporary data
Size Based	Remove items based on memory size	When cache stores variable size items

Advantages and Disadvantages

Aspect	Advantages	Disadvantages
LRU	Simple to implement Good hit ratio	Not ideal for frequency based access May evict frequently used items in some cases
LFU	Keeps frequently used items Higher hit ratio for stable data	Complex to implement Needs additional data structure May not adapt quickly to sudden changes
Other Policies	Useful for specific scenarios Flexible	May not work well for all workloads

Key Takeaways

- Cache eviction is required when cache is full
- LRU removes least recently used item
- LFU removes least frequently used item
- Different policies are useful for different workloads
- Choose the right eviction policy based on access patterns and application needs
- Eviction policies help to maintain high cache hit ratio and better system performance

@Codedsoul_05

14. Distributed Caching + Cache Consistency

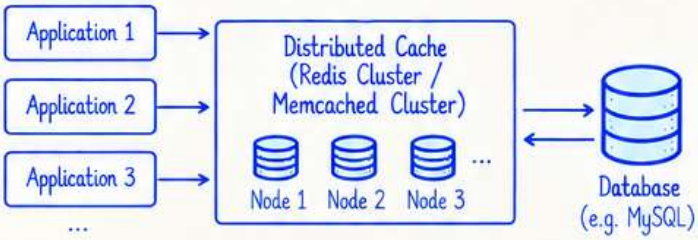
What is Distributed Caching ?

Distributed caching is a caching mechanism where cache data is stored across multiple servers in a distributed system. It helps to handle large scale applications, improve availability, reduce latency and support high traffic by sharing cache data across multiple nodes.

Why Distributed Caching is Needed ?

- Supports high traffic and large data
- Improves availability and fault tolerance
- Reduces load on database servers
- Scales horizontally by adding more cache nodes
- Useful for distributed and microservices architecture
- Helps to share cache data across multiple applications and regions

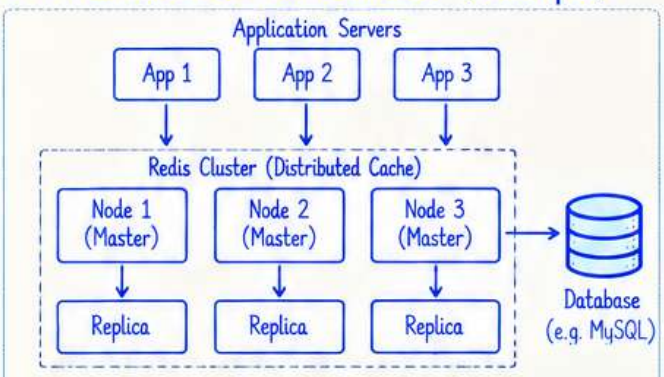
How Distributed Caching Works ?



Common Distributed Caching Solutions

Technology	Type	Features	Use Case
Redis Cluster	In memory	High performance, replication, sharding	Most popular, production systems
Memcached	In memory	Simple, fast, no persistence	Simple caching needs
Hazelcast	In memory	Distributed data grid	Java based systems
Apache Ignite	In memory	In memory computing	Large scale systems
AWS ElastiCache	Managed	Redis / Memcached	Cloud based systems

Architecture : Redis Cluster (Example)



Data is partitioned (sharded) across multiple nodes and replicated for high availability.

Data Partitioning (Sharding)

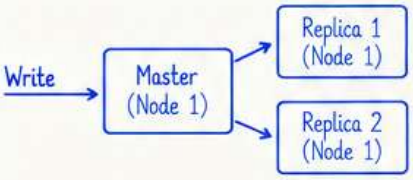
In distributed caching, data is divided into multiple nodes using a hash function.



hash(key) % number_of_nodes

Replication for High Availability

Each master node has one or more replica nodes.



- If master fails, replica can take over
- Data is replicated asynchronously (in Redis by default)
- Provides high availability and fault tolerance

What is Cache Consistency ?

Cache consistency ensures that the data in cache is synchronized with the data in the database. When data changes in the database, the cache should reflect the latest value and should not serve stale data.

Why Cache Consistency is Important ?

- Prevents users from seeing stale or outdated data
- Maintains data correctness across multiple nodes
- Important in distributed systems and microservices
- Helps in applications with frequent data updates

Cache Consistency Patterns

Pattern	Description	How it Works	Use Case
Cache Aside (Lazy Loading)	Application controls cache	Read from cache, if not found fetch from DB, update cache. On write, invalidate cache.	Most common pattern
Read Through	Cache layer fetches data automatically	Cache fetches data from DB on cache miss	When using cache frameworks
Write Through	Write to cache and database together	Data is written to cache and DB simultaneously	When strong consistency needed
Write Back (Write Behind)	Write to cache first, update DB later	Data is written to cache, DB is updated asynchronously	Write heavy applications

Cache Invalidation Strategies

Strategy	Description
Time Based (TTL)	Automatically expire cache after fixed time
Manual Invalidation	Invalidate cache when data changes
Event Based	Invalidate cache using events (e.g. message queue)
Versioning	Use version in cache key (e.g. v1:user:123)
Write Through	Update cache along with database
Write Back	Update cache first, DB later
Cache Refresh	Refresh cache in background before expiry

Challenges in Distributed Caching

Challenge	Description
Cache Consistency	Keeping cache and database in sync
Cache Invalidation	Removing or updating stale data
Network Latency	Latency between application and cache nodes
Node Failure	Handling node failures and data loss
Data Partitioning	Ensuring even distribution of data
Scaling	Managing cache as the system grows
Hot Keys	Some keys are accessed very frequently

Advantages

- ✅ High performance and low latency
- ✅ Reduces database load
- ✅ Scalable and distributed architecture
- ✅ High availability and fault tolerance
- ✅ Supports large datasets
- ✅ Can be used across multiple services
- ✅ Improves overall system performance

Disadvantages

- ❌ Complex to implement and manage
- ❌ Cache consistency is challenging
- ❌ Requires proper invalidation strategy
- ❌ Network overhead
- ❌ Data can be lost if cache fails (depending on configuration)
- ❌ Hot key problem
- ❌ Requires monitoring and maintenance

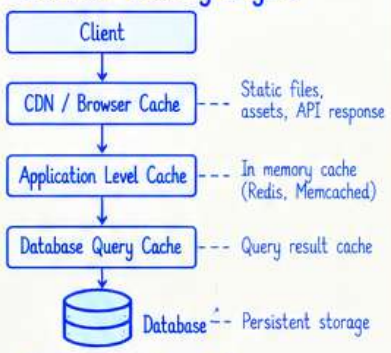
@Codedsoul_05

15. Caching Cheat Sheet + 100 Interview Questions

Caching Cheat Sheet (Quick Notes)

- Caching stores frequently accessed data in faster storage to reduce latency and database load.
- Cache can be in memory (Redis, Memcached) or at different layers (DB, application, API, CDN).
- Common patterns: Cache Aside, Read Through, Write Through, Write Back, Write Around.
- Use TTL and proper invalidation to keep data fresh.
- Important for scalable, high performance systems.

Common Caching Layers



Popular Caching Tools

Tool	Type	Use Case	Key Features
Redis	In memory	General purpose caching	Rich data structures, high performance
Memcached	In memory	Simple caching needs	Simple, fast, lightweight
Hazelcast	In memory	Distributed caching	Data grid, horizontal scaling
Apache Ignite	In memory	Large scale systems	Distributed cache + computing
AWS ElastiCache	Managed	Cloud caching	Managed Redis and Memcached

Caching Patterns

Pattern	How It Works	Use Case
Cache Aside (Lazy Loading)	Application checks cache, if not found fetch from DB	Most common pattern
Read Through	Cache layer fetches data from DB automatically	When using cache frameworks
Write Through	Write to cache and DB together	When strong consistency needed
Write Back (Write Behind)	Write to cache first, update DB later	Write heavy applications
Write Around	Write directly to DB, do not update cache	Data rarely read

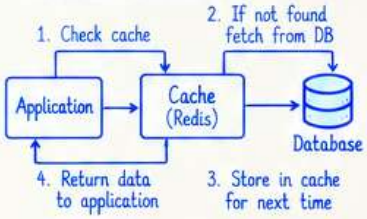
Cache Eviction Policies

Policy	Idea	Use Case
LRU	Remove least recently used	General purpose
LFU	Remove least frequently used	When access frequency matters
FIFO	Remove first inserted	Simple use cases
Random	Remove random item	Simple and fast
TTL Based	Remove expired items	Time sensitive data
Size Based	Remove items based on size	Variable size items

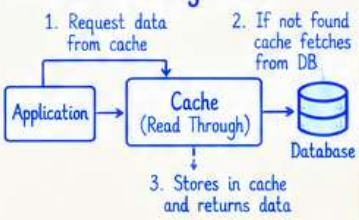
Cache Invalidation Strategies

Strategy	Description	Use Case
Time Based (TTL)	Auto expire after fixed time	Simple and effective
Manual Invalidation	Invalidate on data change	When data updates are known
Event Based	Invalidate using events (e.g. DB update)	Microservices, distributed systems
Versioning	Use version in key (e.g. v1:user:123)	Avoid stale data
Write Through	Update cache and DB	Maintain consistency
Write Back	Update cache first, DB later	Write heavy systems
Cache Refresh	Refresh in background before expiry	Reduce cache miss

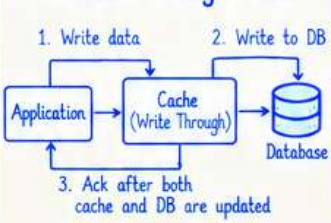
Cache Aside Flow



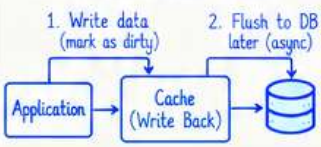
Read Through Flow



Write Through Flow



Write Back Flow



TTL Strategies

Strategy	Description	Example Use Case
Fixed TTL	Same TTL for all entries	5 minutes for product list
Dynamic TTL	Different TTL per data type	Short TTL for price long TTL for categories
Sliding TTL	Extend TTL on each access	Session data
Jittered TTL	Add random time	5 minutes + 30 seconds

Application Level Caching

- Store data in memory inside application.
- Examples: In memory cache, local cache, Caffeine.
- Reduces network calls to external cache.
- Useful for small data or frequent reads.
- Must handle invalidation and memory limits.

Database Query Caching

- Cache the result of frequent queries.
- Reduces database load and latency.
- Can be handled by application or database.
- Use parameterized keys to avoid collisions.
- Invalidate when underlying data changes.

API Caching

- Cache API responses to reduce backend load.
- Use TTL based on data type.
- Can be at application level, reverse proxy (Nginx), or CDN.
- Useful for public data and read heavy APIs.
- Handle cache invalidation on data updates.

Distributed Caching + Consistency

- Share cache data across multiple nodes.
- Use Redis Cluster, Memcached Cluster, Hazelcast.
- Handle node failures and data replication.
- Consistency challenges: cache invalidation, network latency,
- Use strategies like event based invalidation, versioning, and TTL.

Best Practices

- ☑ Choose the right caching layer and pattern
- ☑ Use TTL and proper invalidation
- ☑ Avoid caching sensitive data
- ☑ Use meaningful cache keys with versioning
- ☑ Monitor cache hit ratio and memory usage
- ☑ Handle cache failure gracefully
- ☑ Test for cache consistency in distributed systems

100 Interview Questions

1. What is caching ?
2. Why do we use caching ?
3. What are the benefits of caching ?
4. What are the different types of cache ?
5. What is cache hit and cache miss ?
6. What are the common caching layers ?
7. What is Redis ?
8. What is Memcached ?
9. Redis vs Memcached ?
10. What is application level cache ?
11. What is database query cache ?
12. What is API caching ?
13. What is CDN caching ?
14. What is cache aside pattern ?
15. What is read through caching ?
16. What is write through caching ?
17. What is write back caching ?
18. What is write around caching ?
19. When to use each caching pattern ?
20. Difference between write back and write through ?
21. What is TTL ?
22. Why do we use TTL ?
23. What is sliding TTL ?
24. What is fixed TTL ?
25. What is jittered TTL ?
26. What is cache invalidation ?
27. Why is cache invalidation important ?
28. What are cache invalidation strategies ?
29. What is event based invalidation ?
30. What is versioning in cache keys ?
31. What are cache eviction policies ?
32. What is LRU ?
33. How does LRU work ?
34. What is LFU ?
35. How does LFU work ?
36. What is FIFO eviction ?
37. What is random eviction ?
38. When to use LRU vs LFU ?
39. What is TTL based eviction ?
40. What is size based eviction ?
41. How does Redis store data ?
42. What are Redis data structures ?
43. What is Redis persistence ?
44. What are Redis eviction policies ?
45. What is Redis clustering ?
46. What is Redis replication ?
47. What is Redis sentinel ?
48. What is Memcached architecture ?
49. Why is Memcached faster than Redis ?
50. What are the limitations of Memcached ?
51. How to handle cache consistency ?
52. What is cache consistency in distributed systems ?
53. How to maintain consistency between cache and DB ?
54. What are the challenges of distributed caching ?
55. How to handle cache in microservices ?
56. What is cache stampede ?
57. How to prevent cache stampede ?
58. What is cache penetration ?
59. How to prevent cache penetration ?
60. What is cache avalanche ?
61. How to prevent user session data ?
62. What is cache warming ?
63. What is cache refresh ?
64. What is cache aside with TTL ?
65. What is write around with example ?
66. How to cache large data sets ?
67. How to handle cache for frequently changing data ?
68. How to design cache for high availability ?
69. How to handle cache node failures ?
70. What is Redis cluster vs standalone ?
71. What is consistent hashing ?
72. How to partition cache data ?
73. What is cache key design ?
74. How to avoid key collisions ?
75. What is cache in CDN ?
76. How does browser cache work ?
77. What are cache preders in HTTP ?
78. What is ETag ?
79. What is Cache Control header ?
80. How to handle cache for real time data ?
81. How to cache user session data ?
82. How to cache product catalog data ?
83. How to cache configuration data ?
84. How to cache analytics data ?
85. How to handle cache for writes ?
86. How to invalidate cache after update ?
87. How to monitor cache performance ?
88. What metrics to track in caching ?
89. What is cache hit ratio ?
90. What is cache latency ?
91. What is cache memory usage ?
92. How to scale caching system ?
93. What is multi level caching ?
94. What is local cache vs distributed cache ?
95. How to test caching in system design ?
96. How to choose TTL for different data ?
97. What are best practices for caching ?
98. What are common mistakes in caching ?
99. How to design a caching system for billions of requests ?
100. Design a distributed caching system.

@Codedsoul_05